Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №10
HTTP и REST API в Python: разработка клиент-серверного веб-приложения

1. Цель работы

Изучить принципы работы протокола HTTP и получить практические навыки разработки и использования REST API на Python: разобрать устройство HTTP-запроса и ответа, методы GET, POST, PUT, PATCH и DELETE, заголовки, параметры и коды состояния, освоить принципы REST, разработать собственный HTTP API и клиент для работы с ним, реализовать сериализацию данных в JSON, обработку ошибок и протестировать API.

2. Краткие теоретические сведения

HTTP (HyperText Transfer Protocol) — протокол прикладного уровня для обмена данными между клиентом и сервером. Клиент отправляет запрос, сервер возвращает ответ. HTTP-запрос состоит из строки запроса, заголовков, пустой строки и тела; ответ — из строки состояния, заголовков и тела.

Метод	Назначение	Пример
GET	получение ресурса	GET /rooms/5
POST	создание ресурса	POST /rooms
PUT	полная замена ресурса	PUT /rooms/5
PATCH	частичное изменение	PATCH /rooms/5
DELETE	удаление ресурса	DELETE /rooms/5

Коды состояния показывают результат запроса. Группа 2xx означает успех (200 OK, 201 Created, 204 No Content), 4xx — ошибку клиента (400 Bad Request, 401 Unauthorized, 404 Not Found, 405 Method Not Allowed), 5xx — ошибку сервера (500 Internal Server Error).

REST — архитектурный стиль построения распределённых систем: система состоит из ресурсов, каждый ресурс имеет URI, действия выполняются стандартными HTTP-методами, а взаимодействие является stateless — каждый запрос содержит всю необходимую информацию. URL описывает ресурс, а

метод — действие над ним: GET /rooms, GET /rooms/5, POST /rooms, DELETE /rooms/5.

Данные передаются в формате JSON. Path-параметры входят в путь (/rooms/5) и идентифицируют ресурс, query-параметры передаются после знака вопроса (/rooms?type=люкс&sort=price) и используются для фильтрации, сортировки и пагинации. Заголовок Content-Type описывает тип тела запроса, Accept — ожидаемый тип ответа.

3. Индивидуальное задание

Вариант 5 — предметная область «Отель», ресурс Room (гостиничный номер). Поля: id, number (номер комнаты), room_type (тип), price_per_night (цена за ночь), capacity (количество мест), status (состояние). Специализированная операция варианта — получение свободных номеров: GET /rooms/available.

Требуется реализовать полный набор CRUD-операций, фильтрацию минимум по двум параметрам, сортировку минимум по двум полям, специализированный endpoint, обработку ошибок и Python-клиент с консольным меню.

4. Архитектура приложения

Приложение построено по схеме «клиент — HTTP — REST API — сервис — репозиторий». HTTP-обработчики (routes) только разбирают запрос и вызывают сервис; бизнес-логика и проверки находятся в сервисе; хранение данных скрыто в репозитории. Схема приведена на рисунке 10.1.

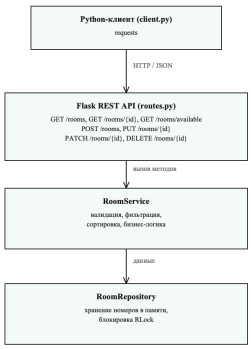


Рисунок 10.1 – Архитектура REST-приложения

Модуль	Назначение
models.py	сущность Room и перечисления RoomType, RoomStatus
repository.py	хранение номеров в памяти, потокобезопасные операции
schemas.py	валидация входных данных, разбор query-параметров
service.py	бизнес-логика: CRUD, фильтрация, сортировка, свободные номера
routes.py	HTTP-обработчики REST API
app.py	создание Flask-приложения, обработчики ошибок
client.py	консольный клиент на requests
exceptions.py	ApiError, ValidationError, InvalidIdError, RoomNotFoundError

5. Описание API

Все endpoints возвращают JSON. Ошибки имеют единый формат: {"error": "текст", "code": "КОД_ОШИБКИ"}.

Метод	Endpoint	Назначение	Успешный код
GET	/rooms	список номеров (фильтры, сортировка)	200
GET	/rooms/{id}	номер по идентификатору	200
POST	/rooms	создание номера	201
PUT	/rooms/{id}	полное изменение номера	200

Метод	Endpoint	Назначение	Успешный код
PATCH	/rooms/{id}	частичное изменение номера	200
DELETE	/rooms/{id}	удаление номера	204
GET	/rooms/available	свободные номера	200

Фильтрация: type, status, min_price, max_price. Сортировка: sort=price|number|capacity и order=asc|desc. Параметры передаются в query-строке, идентификатор — в пути.

6. Примеры HTTP-запросов

Получение списка номеров:

```
GET /rooms HTTP/1.1
Host: 127.0.0.1:5000
Accept: application/json

HTTP/1.1 200 OK
[{"id": 1, "number": "101", "room_type": "стандартный", ...}]
```

Создание номера:

```
POST /rooms HTTP/1.1
Content-Type: application/json

{"number": "105", "room_type": "люкс", "price_per_night": 6500, "capacity": 2}

HTTP/1.1 201 CREATED
Location: /rooms/5
{"id": 5, "number": "105", "room_type": "люкс", "status": "свободен", ...}
```

Частичное изменение и удаление:

```
PATCH /rooms/5
{"price_per_night": 6100}
HTTP/1.1 200 OK

DELETE /rooms/5
HTTP/1.1 204 NO CONTENT
```

Ошибки API:

```
GET /rooms/999   -> 404  {"error": "Номер с ID 999 не найден", "code":
"ROOM_NOT_FOUND"}
GET /rooms/abc   -> 400  {"error": "Идентификатор должен быть целым числом",
"code": "INVALID_ID"}
POST /rooms      -> 400  {"error": "Поле number не может быть пустым", "code":
"VALIDATION_ERROR"}
```

7. Реализация

7.1. Модель данных

Файл: models.py

```
from dataclasses import dataclass
from enum import Enum
from typing import Any, Dict


class RoomType(Enum):
    STANDARD = "стандартный"
```

```
    SUITE = "люкс"
    FAMILY = "семейный"


class RoomStatus(Enum):
    FREE = "свободен"
    OCCUPIED = "занят"
    MAINTENANCE = "на обслуживании"


# Ресурс REST API — гостиничный номер.
# Идентификатор и пять дополнительных полей.
@dataclass
class Room:
    id: int
    number: str
    room_type: RoomType
    price_per_night: float
    capacity: int
    status: RoomStatus

    def to_dict(self) -> Dict[str, Any]:
        return {
            "id": self.id,
            "number": self.number,
            "room_type": self.room_type.value,
            "price_per_night": self.price_per_night,
            "capacity": self.capacity,
            "status": self.status.value,
        }
```

7.2. Репозиторий

Flask может обрабатывать несколько запросов параллельно, поэтому операции репозитория защищены блокировкой RLock.

Файл: repository.py

```
from threading import RLock
from typing import Dict, List, Optional

from models import Room


# Хранилище номеров в памяти процесса. Flask может обслуживать
# несколько запросов одновременно, поэтому операции защищены блокировкой.
class RoomRepository:
    def __init__(self):
        self._rooms: Dict[int, Room] = {}
        self._lock = RLock()

    def get_all(self) -> List[Room]:
        with self._lock:
            return list(self._rooms.values())

    def get_by_id(self, room_id: int) -> Optional[Room]:
        with self._lock:
            return self._rooms.get(room_id)

    def create(self, room: Room) -> Room:
        with self._lock:
            if room.id in self._rooms:
                raise ValueError(f"Номер с ID {room.id} уже существует")
            self._rooms[room.id] = room
            return room

    def update(self, room: Room) -> Room:
        with self._lock:
            if room.id not in self._rooms:
                raise ValueError(f"Номер с ID {room.id} не найден")
```

```
            self._rooms[room.id] = room
            return room

    def delete(self, room_id: int) -> bool:
        with self._lock:
            return self._rooms.pop(room_id, None) is not None

    def next_id(self) -> int:
        with self._lock:
            return max(self._rooms, default=0) + 1
```

7.3. Сервис

Сервис выполняет валидацию, фильтрацию, сортировку и не зависит от HTTP. Некорректные данные приводят к ValidationError, отсутствующий номер — к RoomNotFoundError.

Файл: service.py

```
from typing import Any, Dict, List

from exceptions import RoomNotFoundError
from models import Room, RoomStatus
from repository import RoomRepository
from schemas import SORT_FIELDS, parse_room_payload


# Бизнес-логика работы с номерами. Сервис не знает ни про HTTP,
# ни про Flask: он получает данные и возвращает объекты Room.
class RoomService:
    def __init__(self, repository: RoomRepository):
        self._repository = repository

    # Список номеров с фильтрацией и сортировкой (query-параметры)
    def list_rooms(self, filters: Dict[str, Any]) -> List[Room]:
        rooms = self._repository.get_all()

        if "room_type" in filters:
            rooms = [room for room in rooms if room.room_type ==
filters["room_type"]]
        if "status" in filters:
            rooms = [room for room in rooms if room.status == filters["status"]]
        if "min_price" in filters:
            rooms = [room for room in rooms if room.price_per_night >=
filters["min_price"]]
        if "max_price" in filters:
            rooms = [room for room in rooms if room.price_per_night <=
filters["max_price"]]

        sort = filters.get("sort")
        if sort:
            field = SORT_FIELDS[sort]
            rooms.sort(key=lambda room: getattr(room, field),
reverse=filters.get("order") == "desc")

        return rooms

    def get_room(self, room_id: int) -> Room:
        room = self._repository.get_by_id(room_id)
        if room is None:
            raise RoomNotFoundError(room_id)
        return room

    def create_room(self, payload: Any) -> Room:
        data = parse_room_payload(payload)
        room = Room(
            id=self._repository.next_id(),
            number=data["number"],
```

```
            room_type=data["room_type"],
            price_per_night=data["price_per_night"],
            capacity=data["capacity"],
            status=data.get("status", RoomStatus.FREE),
        )
        return self._repository.create(room)

    # PUT: полная замена ресурса, все основные поля обязательны
    def replace_room(self, room_id: int, payload: Any) -> Room:
        current = self.get_room(room_id)
        data = parse_room_payload(payload)
        room = Room(
            id=current.id,
            number=data["number"],
            room_type=data["room_type"],
            price_per_night=data["price_per_night"],
            capacity=data["capacity"],
            status=data.get("status", current.status),
        )
        return self._repository.update(room)

    # PATCH: частичное изменение, достаточно одного поля
    def patch_room(self, room_id: int, payload: Any) -> Room:
        room = self.get_room(room_id)
        changes = parse_room_payload(payload, partial=True)
        for field, value in changes.items():
            setattr(room, field, value)
        return self._repository.update(room)

    def delete_room(self, room_id: int) -> None:
        if not self._repository.delete(room_id):
            raise RoomNotFoundError(room_id)

    # Специализированная операция варианта: свободные номера
    def get_available_rooms(self, filters: Dict[str, Any]) -> List[Room]:
        filters = dict(filters)
        filters["status"] = RoomStatus.FREE
        return self.list_rooms(filters)
```

7.4. Валидация и query-параметры

Файл: schemas.py

```
from typing import Any, Dict, Mapping

from exceptions import InvalidIdError, ValidationError
from models import RoomStatus, RoomType

ROOM_TYPES = {item.value: item for item in RoomType}
ROOM_STATUSES = {item.value: item for item in RoomStatus}
SORT_FIELDS = {"price": "price_per_night", "number": "number", "capacity":
"capacity"}
SORT_ORDERS = ("asc", "desc")


def parse_id(value: Any) -> int:
    try:
        room_id = int(str(value))
    except (TypeError, ValueError):
        raise InvalidIdError()
    if room_id <= 0:
        raise InvalidIdError("Идентификатор должен быть положительным целым
числом")
    return room_id


def parse_json_object(payload: Any) -> Dict[str, Any]:
    if not isinstance(payload, dict):
        raise ValidationError("Тело запроса должно быть JSON-объектом")
    return payload


def parse_room_type(value: Any) -> RoomType:
```

```
    room_type = ROOM_TYPES.get(str(value).strip().lower())
    if room_type is None:
        allowed = ", ".join(ROOM_TYPES)
        raise ValidationError(f"Недопустимый тип номера, ожидается одно из:
{allowed}")
    return room_type


def parse_status(value: Any) -> RoomStatus:
    status = ROOM_STATUSES.get(str(value).strip().lower())
    if status is None:
        allowed = ", ".join(ROOM_STATUSES)
        raise ValidationError(f"Недопустимый статус, ожидается одно из:
{allowed}")
    return status


def parse_number(value: Any) -> str:
    number = str(value).strip() if value is not None else ""
    if not number:
        raise ValidationError("Поле number не может быть пустым")
    return number


def parse_price(value: Any) -> float:
    try:
        price = float(value)
    except (TypeError, ValueError):
        raise ValidationError("Поле price_per_night должно быть числом")
    if price <= 0:
        raise ValidationError("Поле price_per_night должно быть больше 0")
    return round(price, 2)
```

Остальные функции модуля проверяют отдельные поля: тип номера, статус, цену, количество мест и идентификатор.

7.5. HTTP-обработчики

Каждый обработчик состоит из одной-двух строк: разобрать параметры, вызвать сервис, вернуть JSON и HTTP-код. Бизнес-логика в маршрутах отсутствует.

Файл: routes.py

```
from flask import Blueprint, jsonify, request

from schemas import parse_filters, parse_id
from service import RoomService


# HTTP-уровень: разбор запроса, вызов сервиса, формирование ответа.
# Бизнес-логика находится в RoomService.
def create_blueprint(service: RoomService) -> Blueprint:
    bp = Blueprint("rooms", __name__)

    # Специализированный endpoint объявлен раньше динамического,
    # чтобы слово available не воспринималось как идентификатор.
    @bp.get("/rooms/available")
    def available_rooms():
        filters = parse_filters(request.args)
        rooms = service.get_available_rooms(filters)
        return jsonify([room.to_dict() for room in rooms]), 200
```

```
    @bp.get("/rooms")
    def list_rooms():
        filters = parse_filters(request.args)
        rooms = service.list_rooms(filters)
        return jsonify([room.to_dict() for room in rooms]), 200

    @bp.get("/rooms/<room_id>")
    def get_room(room_id: str):
        room = service.get_room(parse_id(room_id))
        return jsonify(room.to_dict()), 200

    @bp.post("/rooms")
    def create_room():
        room = service.create_room(request.get_json(silent=True))
        response = jsonify(room.to_dict())
        response.headers["Location"] = f"/rooms/{room.id}"
        return response, 201

    @bp.put("/rooms/<room_id>")
    def replace_room(room_id: str):
        room = service.replace_room(parse_id(room_id),
request.get_json(silent=True))
        return jsonify(room.to_dict()), 200

    @bp.patch("/rooms/<room_id>")
    def patch_room(room_id: str):
        room = service.patch_room(parse_id(room_id),
request.get_json(silent=True))
        return jsonify(room.to_dict()), 200

    @bp.delete("/rooms/<room_id>")
    def delete_room(room_id: str):
        service.delete_room(parse_id(room_id))
        return "", 204

    return bp
```

7.6. Приложение и обработка ошибок

Собственные исключения содержат HTTP-код и машиночитаемый код ошибки, поэтому обработчик ApiError автоматически формирует корректный ответ. Дополнительно обрабатываются 404 для неизвестного маршрута, 405 для неподдерживаемого метода и 500 для непредвиденных ошибок.

Файл: app.py

```
import logging

from flask import Flask, jsonify

from exceptions import ApiError
from repository import RoomRepository
from routes import create_blueprint
from service import RoomService

SEED_ROOMS = [
    {"number": "101", "room_type": "стандартный", "price_per_night": 2500,
"capacity": 2},
    {"number": "102", "room_type": "люкс", "price_per_night": 6000, "capacity":
2},
    {"number": "103", "room_type": "семейный", "price_per_night": 4200,
"capacity": 4, "status": "занят"},
    {"number": "104", "room_type": "стандартный", "price_per_night": 2200,
"capacity": 2, "status": "на обслуживании"},
]
```

```
def create_app(repository: RoomRepository = None, seed: bool = True) -> Flask:
    app = Flask(__name__)
    app.json.ensure_ascii = False

    repository = repository or RoomRepository()
    service = RoomService(repository)
    app.extensions["room_service"] = service
    app.register_blueprint(create_blueprint(service))

    if seed and not repository.get_all():
        for payload in SEED_ROOMS:
            service.create_room(payload)

    @app.errorhandler(ApiError)
    def handle_api_error(error: ApiError):
        return jsonify(error.to_dict()), error.status_code

    @app.errorhandler(404)
    def handle_not_found(error):
        return jsonify({"error": "Ресурс не найден", "code": "NOT_FOUND"}), 404

    @app.errorhandler(405)
    def handle_method_not_allowed(error):
        return jsonify({"error": "Метод не поддерживается", "code":
"METHOD_NOT_ALLOWED"}), 405

    @app.errorhandler(Exception)
    def handle_internal_error(error: Exception):
        logging.getLogger("room_api").exception("Unhandled error: %s", error)
        return jsonify({"error": "Внутренняя ошибка сервера", "code":
"INTERNAL_ERROR"}), 500

    return app


app = create_app()


if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000)
```

7.7. Клиент

Файл: client.py

```
import sys

import requests

BASE_URL = "http://127.0.0.1:5000"
TIMEOUT = 5
ROOM_TYPES = ("стандартный", "люкс", "семейный")
ROOM_STATUSES = ("свободен", "занят", "на обслуживании")


def print_rooms(rooms) -> None:
    if not rooms:
        print("Номеров нет.")
        return
    print(f"{'ID':>3}  {'Номер':<6} {'Тип':<12} {'Цена':>9} {'Мест':>4}
Статус")
    for room in rooms:
        print(f"{room['id']:>3}  {room['number']:<6} {room['room_type']:<12} "
              f"{room['price_per_night']:>9.2f} {room['capacity']:>4}
{room['status']}")


def show(response: requests.Response) -> None:
    if response.status_code == 204:
        print(f"HTTP {response.status_code}: запрос выполнен, тело ответа
отсутствует.")
        return
    try:
```

```
        payload = response.json()
    except ValueError:
        print(f"HTTP {response.status_code}: некорректный ответ сервера")
        return

    if response.status_code >= 400:
        print(f"HTTP {response.status_code}: {payload.get('error')} (код
{payload.get('code')})")
        return

    if isinstance(payload, list):
        print(f"HTTP {response.status_code}: получено {len(payload)} записей")
        print_rooms(payload)
    else:
        print(f"HTTP {response.status_code}:")
        print_rooms([payload])


def read_room_payload(partial: bool = False) -> dict:
    payload = {}
    number = input("Номер комнаты: ").strip()
    if number or not partial:
        payload["number"] = number
    room_type = input(f"Тип ({'/'.join(ROOM_TYPES)}): ").strip()
    if room_type or not partial:
        payload["room_type"] = room_type
    price = input("Цена за ночь: ").strip()
    if price or not partial:
        payload["price_per_night"] = price
    capacity = input("Количество мест: ").strip()
    if capacity or not partial:
        payload["capacity"] = capacity
    status = input(f"Статус ({'/'.join(ROOM_STATUSES)}), Enter — не менять:
").strip()
    if status:
        payload["status"] = status
    return {key: value for key, value in payload.items() if value != ""}


def main() -> None:
    session = requests.Session()
    session.headers.update({"Accept": "application/json"})

    while True:
        print("\n==========================")
        print("       REST CLIENT")
        print("==========================")
        print("1. Получить список")
        print("2. Получить объект")
        print("3. Создать")
        print("4. Изменить (PUT)")
        print("5. Частично изменить (PATCH)")
```

Клиент использует библиотеку requests и консольное меню из десяти пунктов: список, получение объекта, создание, PUT, PATCH, удаление, фильтрация, сортировка, свободные номера и выход. Ошибки сервера выводятся вместе с HTTP-кодом и полем code.

8. Тестирование

Написано 23 автоматических теста на pytest: четыре теста репозитория, шесть тестов сервиса и тринадцать тестов API. Тесты API используют test_client Flask и проверяют список, получение объекта, создание, полное и частичное

изменение, удаление, ошибки 400 и 404, фильтрацию, сортировку, специализированный endpoint и 405 Method Not Allowed. Результат выполнения приведён на рисунке 10.2.

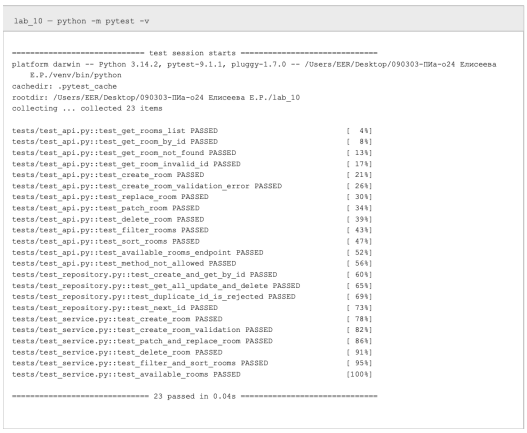


Рисунок 10.2 – Результаты выполнения pytest

9. Демонстрация работы

Тест 1. Запуск Flask-сервера и журнал обработанных запросов (рисунок 10.3).

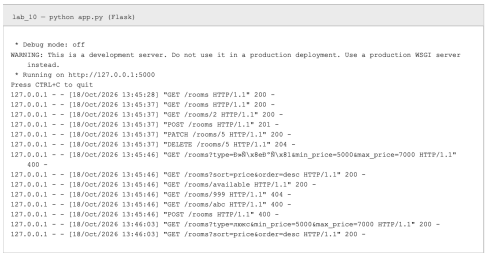


Рисунок 10.3 – Запуск сервера и журнал запросов

Тест 2. Основные HTTP-операции: получение списка, получение объекта, создание (201 Created с заголовком Location), частичное изменение и удаление (204 No Content) — рисунок 10.4.



Рисунок 10.4 – Основные HTTP-запросы

Тест 3. Фильтрация, сортировка, свободные номера и обработка ошибок 404/400 (рисунок 10.5).



Рисунок 10.5 – Фильтрация, сортировка и ошибки

Тест 4. Работа Python-клиента: список номеров, получение объекта, создание и частичное изменение (рисунок 10.6).



Рисунок 10.6 – Работа Python-клиента

Тест 5. Фильтрация и сортировка через клиент, а также специализированная операция «свободные номера» (рисунок 10.7).

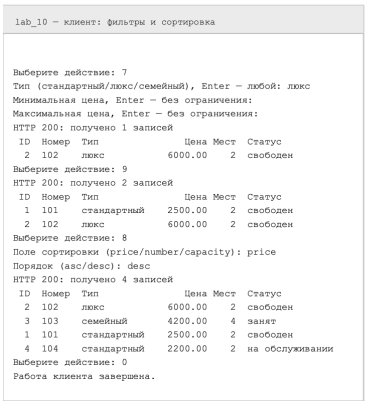


Рисунок 10.7 – Фильтрация, сортировка и свободные номера

9. Вывод

В ходе лабораторной работы было разработано REST API гостиницы на Flask: полный набор CRUD-операций для ресурса Room, фильтрация по типу, статусу и диапазону цены, сортировка по цене, номеру и количеству мест, специализированный endpoint /rooms/available и единый формат ошибок с корректными HTTP-кодами.

Разделение на маршруты, сервис и репозиторий позволило держать HTTP-обработчики минимальными и протестировать бизнес-логику отдельно от сети. Клиент на requests повторяет все операции API и показывает коды ответов. В результате получены практические навыки проектирования REST API, работы с HTTP-методами, параметрами, заголовками и кодами состояния.